

Техническое задание
ProjectHub — Система управления проектами

Муратов Андрей Сергеевич

12 апреля 2026 г.

Содержание

1	Название проекта	3
2	Описание проекта	3
3	Сущности, примеры сущностей	3
4	Функциональные требования	3
5	Безопасность	4
6	Интеграции и инструменты	5
7	Базы данных	5
8	Тестирование	5
9	Архитектура	5
10	Стек технологий	6
11	Документация (планируемая)	6
12	Распределение задач между членами команды	6
13	Формат сдачи проекта	7

1 Название проекта

ProjectHub — Система управления проектами и задачами

2 Описание проекта

Веб-приложение с графическим интерфейсом для создания проектов, управления задачами, назначения участников и контроля сроков выполнения. Приложение обеспечивает централизованный учёт рабочих процессов, разделение прав доступа между пользователями и администраторами, а также ведение истории изменений через комментарии. Интерфейс реализован в виде веб-страниц, открываемых в браузере, и полностью содержит базовые элементы GUI: формы ввода, таблицы данных и систему навигации.

3 Сущности, примеры сущностей

Сущность	Поля	Пример
User	id, login, password_hash, role, created_at	{id: 1, login: "ivan_dev role: "USER"}
Project	id, title, description, owner_id, status, created_at	{id: 10, title: "Миграция БД status: "ACTIVE"}
Task	id, title, description, status, deadline, project_id, assignee_id, created_at	{id: 101, title: "Написать SQL status: "IN_PROGRESS"}
Comment	id, text, created_at, task_id, author_id	{id: 305, text: "Готово, прошу проверить"}

Связи: User $1 \rightarrow \infty$ Project, Project $1 \rightarrow \infty$ Task, Task $1 \rightarrow \infty$ Comment, User $1 \rightarrow \infty$ Comment.

4 Функциональные требования

Пользователь (ROLE_USER)

- Авторизация в системе по логину и паролю.
- Просмотр списка созданных им проектов.
- Создание, редактирование и удаление проектов (только своих).
- Просмотр задач внутри проекта в табличном виде.
- Создание, редактирование, изменение статуса задач.

- Добавление комментариев к задачам.
- Фильтрация и сортировка задач по статусу и дедлайну.

Администратор (ROLE_ADMIN)

- Все функции пользователя.
- Просмотр и управление всеми проектами в системе.
- Просмотр списка зарегистрированных пользователей.
- Назначение/изменение ролей пользователей.
- Просмотр сводной статистики (общее кол-во проектов/задач).

Интерфейс (GUI)

- Реализованы формы ввода данных (создание/редактирование сущностей).
- Реализованы таблицы с отображением списков проектов/задач/пользователей.
- Реализована навигация: меню переходов между разделами, хлебные крошки, пагинация.

5 Безопасность

- Аутентификация и авторизация через Spring Security.
- Пароли пользователей хранятся в БД исключительно в зашифрованном виде (BCryptPasswordEncoder).
- Разграничение прав доступа на основе ролей (RBAC): проверка прав на уровне URL-маршрутов и методов сервисов.
- Валидация входящих данных на уровне форм и сервисного слоя (@Valid, ручные проверки).
- Защита от CSRF и базовых web-уязвимостей средствами Spring Security.
- Обработка исключений и graceful fallback для обеспечения устойчивости работы без критических сбоев.

6 Интеграции и инструменты

- Spring Boot 3.2+ (основа приложения)
- Spring Security (безопасность)
- Spring Data JPA (доступ к данным)
- Thymeleaf + Bootstrap 5 (графический интерфейс)
- Maven (сборка и управление зависимостями)
- Git + GitHub (система контроля версий)
- Docker / Docker Compose (опционально, для контейнеризации)

7 Базы данных

- Тип: реляционная СУБД PostgreSQL 14+.
- ORM: Hibernate (через абстракцию Spring Data JPA).
- Реализованы сущности (@Entity) и репозитории (JpaRepository) для всех бизнес-сущностей.
- Схема БД нормализована до 3НФ. Связи реализованы через @OneToMany, @ManyToOne.
- Инициализация схемы: spring.jpa.hibernate.ddl-auto=update (для ускорения разработки, опционально заменяется на Flyway/Liquibase).

8 Тестирование

- Unit-тесты: покрытие сервисного слоя с использованием JUnit 5 + Mockito (CRUD задач, валидация, авторизация).
- Интеграционные тесты: проверка репозитория через Testcontainers (PostgreSQL) или встроенную H2.
- Ручное тестирование: проверка всех GUI-сценариев перед записью демо-видео.
- Примечание: Тестирование не блокирует сдачу, но входит в план повышения итоговой оценки.

9 Архитектура

Трёхслойная архитектура с чётким разделением ответственности:

1. Presentation Layer: @Controller + Thymeleaf-шаблоны. Отвечает за обработку HTTP-запросов, рендеринг GUI и валидацию форм.

2. Business Logic Layer: @Service. Содержит бизнес-правила, транзакционное управление (@Transactional), DTO-маппинг и проверку прав.
 3. Data Access Layer: @Repository. Наследует JpaRepository, выполняет запросы к PostgreSQL через Hibernate.
- Использование паттерна DTO для передачи данных между слоями.
 - Обработка ошибок через @ControllerAdvice и кастомные Exception-классы.

10 Стек технологий

- Язык: Java 17+
- Фреймворк: Spring Boot 3.2 (Web, Security, Data JPA, Validation)
- ORM: Hibernate / JPA
- БД: PostgreSQL 14+
- GUI: Thymeleaf, HTML5, CSS3, Bootstrap 5
- Логирование: SLF4J + Logback (встроено в Spring Boot)
- Сборка: Maven
- Дополнительно: REST API, Swagger/OpenAPI, Docker, JUnit 5

11 Документация (планируемая)

- JavaDoc: описание всех публичных классов, методов и констант.
- README.md: инструкция по клонированию, настройке БД, сборке, запуску, демо-данным для входа, описание архитектуры и скриншоты интерфейса.
- Swagger UI (опционально): автоматическая документация REST-эндпоинтов.
- Видео-демонстрация: запись работы приложения (2–3 минуты) по сценарию из ТЗ.

12 Распределение задач между членами команды

Проект выполняется индивидуально (1 участник). Все этапы (проектирование БД, реализация слоёв, настройка безопасности, создание GUI, написание тестов, документация и запись демо) выполняются одним разработчиком.

13 Формат сдачи проекта

Проект сдаётся в виде папки, загруженной в облачное хранилище. Папка содержит:

- Файл с данным ТЗ
- Ссылка на репозиторий с кодом (GitHub)
- Ссылка на видео с демонстрацией работы приложения (2-3 минуты)
- Документация приложения (README.md, JavaDoc, Swagger)